

Práctica 2. RRNN y Cálculo Paralelo

Siria Catherine Iñiguez Brito

12 de abril de 2026

1. Ejemplo 2 y Ejercicio 2a

Se estudia la evaluación de la función externa `func1.m` para un vector de datos. Un aspecto crítico en la computación paralela es la relación entre el tiempo de cómputo y el *overhead* de comunicación. Para analizar este fenómeno, se ejecutó el script variando la carga de trabajo mediante diferentes valores en la instrucción `pause`.

1.1. Resultados para Ejemplo 2

A continuación se presentan los tiempos obtenidos (en segundos) y el *Speedup* resultante para 6 *workers* en función del retardo aplicado:

Cuadro 1: Comparativa de tiempos y Speedup variando el parámetro `pause`

Retardo	Secuencial (<i>for</i>)	Secuencial (<i>array</i>)	Paralelo (<i>parfor</i>)	Speedup
Sin pause	0.01216	0.001131	0.21664	0.055913
pause(0.001)	0.059805	0.038844	0.4241	0.14036
pause(0.01)	0.60424	0.61536	0.33644	1.7962
pause(0.1)	3.871	3.8527	0.9066	4.2699
pause(0.2)	7.4205	7.3922	1.4402	5.1524
pause(0.5)	18.2011	18.2414	3.2499	5.6013

Cuando el retardo es nulo o muy bajo, el tiempo que tarda MATLAB en gestionar los hilos y enviar los datos a los *workers* supera el beneficio del cálculo en paralelo, resultando en un *speedup* pobre. A medida que el valor del `pause` aumenta, la carga computacional de cada iteración se vuelve mucho más pesada que el coste de comunicación. Es en estos escenarios donde la paralelización ofrece su rendimiento óptimo, acercándose a un aprovechamiento real de los 6 núcleos de procesamiento.

1.2. Ejercicio 2a: Evaluación paralela con `arrayfun`

En este ejercicio se propone realizar una versión paralela del código para la evaluación de la función `func1` utilizando el comando `arrayfun`. Con el fin de estudiar a fondo su comportamiento frente a los métodos tradicionales, se repitieron las pruebas variando el retardo (`pause`) desde 0 hasta 0.5 segundos.

1.3. Resultados y Análisis Comparativo

En la Tabla 2 se recogen los tiempos de ejecución obtenidos con los cuatro métodos planteados.

- **Sin pause y 0.001):** El uso de vectores en secuencial (`arrayfun`) es la opción más rápida por muchísima diferencia. Intentar paralelizar estas tareas tan sumamente cortas genera pérdidas de tiempo debido al *overhead* de comunicación que requiere gestionar los 6 *workers*.
- **Entre 0.1 y 0.5):** A medida que el cómputo se vuelve costoso, el paralelismo empieza a dominar. El método `arrayfun` en paralelo demuestra ser más rápido que el `parfor` tradicional.

Cuadro 2: Tiempos de ejecución (s) para el Ejercicio 2a

Retardo	Secuencial (<i>for</i>)	Secuencial (<i>array</i>)	Paralelo (<i>for</i>)	Paralelo (<i>array</i>)
Sin pause	0.011661	0.0009074	0.22283	0.055982
pause(0.001)	0.052617	0.039731	0.27896	0.063439
pause(0.01)	0.58136	0.58266	0.29336	0.13814
pause(0.1)	3.8908	3.9076	0.8727	0.70612
pause(0.2)	7.4312	7.4131	1.4836	1.2945
pause(0.5)	18.2884	18.2887	3.2887	3.105

2. Análisis de Función Directa (directfunc.m)

En este apartado se evalúa la misma expresión matemática pero de forma analítica directa (vectorizada) sobre un volumen masivo de datos ($4 \times 12\,000\,000$ de elementos), comparándola con una repartición por bloques en un bucle `parfor`.

Cuadro 3: Tiempos de ejecución con función directa

Método	Tiempo (s)	Speedup
Directo Secuencial	1.1598	-
Directo Paralelo (6 workers)	32.871	0.035281

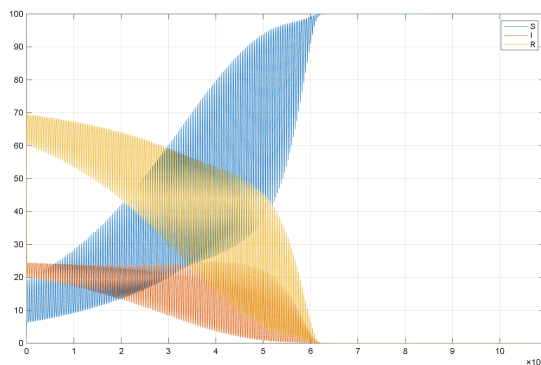
MATLAB maneja de forma nativa operaciones vectorizadas. Al forzar un bucle `parfor`, el coste temporal de la transferencia y partición de datos masivos supera con creces el tiempo de cálculo, resultando en una pérdida masiva de eficiencia.

3. Ejercicio 2b: Modelo Epidemiológico e Incertidumbres

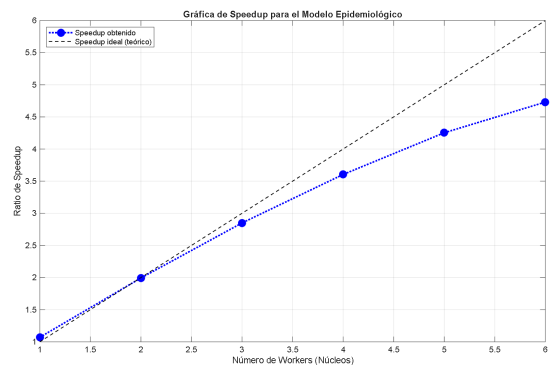
Se propone paralelizar un código de simulación Montecarlo para evaluar las incertidumbres de un modelo epidemiológico que resuelve un sistema de ecuaciones diferenciales ordinarias con `ode23`.

La Figura 1a muestra la evolución temporal del modelo para las poblaciones Susceptibles (S), Infectados (I) y Recuperados (R). La Figura 1b presenta la curva de *Speedup* obtenida al variar el número de núcleos de 1 a 6.

El comportamiento del Speedup es muy cercano a la línea ideal hasta un número de *workers* igual a 3. Después se va distanciando, pero aún así alcanza un valor de **4.7x** con 6 *workers*.



(a) Modelo S-I-R.



(b) Speedup vs Ideal.

Figura 1: Resultados de la simulación y análisis de rendimiento.